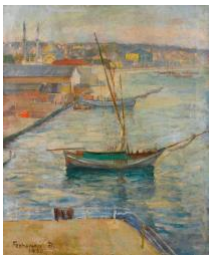
Kerem Kaya - 33917

Painting and transformations



I chose Feyhaman Duran - Kabataş Rıhtımı (1936). The curved lower edge of the foreground boat gives me a clear shape to trace. The waterfront and the boats also make it easy to compare a flat painting with objects placed in front of it. I draw the painting as a textured quad and place three cubes in front of it.

I use plain JavaScript, HTML, CSS, WebGPU and WGSL. Vite serves the application. Matrix, camera and curve calculations are kept in separate modules.

Part A - Transformation order

I store matrices column-major and multiply column vectors: p_clip = P * V * M * p_local. TRS applies scale, rotation and translation in that order. SRT translates first, so the later rotation and scale also change the translation. The orange cube has controls for translation, Z rotation and uniform scale; the other objects use TRS.

For T = (1, 0, 0.8), Rz = 30 degrees and S = 0.6, I obtain:

TRS

0.519615	-0.300000	0.000000	1.000000
0.300000	0.519615	0.000000	0.000000
0.000000	0.000000	0.600000	0.800000
0.000000	0.000000	0.000000	1.000000

SRT

0.519615	-0.300000	0.000000	0.519615
0.300000	0.519615	0.000000	0.300000
0.000000	0.000000	0.600000	0.480000
0.000000	0.000000	0.000000	1.000000

For a rigid transform M = [R t; 0 1], I use $R^{-1} = R^T$, giving $M^{-1} = [R^T \; -R^T t; \; 0 \; 1]$. I do not use this shortcut on matrices containing scale. I also implemented identity, multiply, translate, all three axis rotations, scale and transpose.



T*R*S

0.520	-0.300	0.000	1.000
0.300	0.520	0.000	0.000
0.000	0.000	0.600	0.800
0.000	0.000	0.000	1.000

S*R*T

0.520	-0.300	0.000	0.520
0.300	0.520	0.000	0.300
0.000	0.000	0.600	0.480
0.000	0.000	0.000	1.000

TRS, perspective. Scene and order/matrix readout from the same screenshot.



T*R*S

0.520	-0.300	0.000	1.000
0.300	0.520	0.000	0.000
0.000	0.000	0.600	0.800
0.000	0.000	0.000	1.000

S*R*T

0.520	-0.300	0.000	0.520
0.300	0.520	0.000	0.300
0.000	0.000	0.600	0.480
0.000	0.000	0.000	1.000

SRT, perspective. Scene and order/matrix readout from the same screenshot.

The numerical results show the difference directly: TRS leaves the center at (1, 0, 0.8), whereas SRT moves it to about (0.519615, 0.3, 0.48). Both matrices remain visible when I switch the applied order.

Scene graph and camera

Part B - Hierarchy and a leaf world matrix

I traverse from the root and compute world = parent.world * local. Rotating the pivot moves its child with it. The debug axes use each node's world transform.

```
gallery +--
animated-pivot
|   +-- lavender-cube
+-- amber-cube
+-- teal-cube
```

```
At pivot angle 0:
 0.5  0    0    0
0    0.5  0    0.8
 0    0    0.5  0.9
 0    0    0    1
```

By hand: I * T(-0.8, 0.8, 0.9) * Rz(0) * T(0.8, 0, 0) * S(0.5) = T(0, 0.8, 0.9) * S(0.5). Thus local (1, 0, 0) becomes (0.5, 0.8, 0.9). At 90 degrees the cube center is (-0.8, 1.6, 0.9). The zero-angle button lets me check the first result on screen.

Part C - View and projection

I form a right-handed camera basis with two cross products. For a world point p, the coordinates are dot(r, p-eye), dot(u, p-eye) and dot(b, p-eye), which gives the view matrix below. The eye maps to the origin.

```
b       = normalize(eye-
target)r =
normalize(cross(up,b)) u =
cross(b,r) V =
 r.x r.y r.z -dot(r,eye)
 u.x u.y u.z -dot(u,eye)
 b.x b.y b.z -dot(b,eye)
 0   0   0    1
```

```
s = 1/tan(fovY/2), n=near, F=far
w_clip = -z; z_clip = a*z+b (-
a*n+b)/n = 0
(-a*F+b)/F = 1
=> a=F/(n-F), b=n*F/(n-
F) P =  s/aspect 0  0
0  0        s  0
0
 0        0  F/(n-F)    n*F/(n-F)
 0        0 -1          0
```

If up is parallel to the view direction, I choose another up axis before taking the cross product. A zero up vector is handled the same way; coincident eye and target use b = (0, 0, 1).

For orthographic projection I solve a linear mapping instead: z_ndc = (z+n)/(n-F). Therefore -n maps to 0 and -F to 1. The X and Y mappings are (2x-r-l)/(r-l) and (2y-t-b)/(t-b); w stays 1. This gives the same WebGPU depth endpoints as perspective.

```
P_ortho =
 2/(r-l)  0        0          -(r+l)/(r-l)
 0        2/(t-b)  0          -(t+b)/(t-b)
 0        0        1/(n-F)     n/(n-F)
 0        0        0           1
```

I use near = 0.5 and far = 16, a ratio of 32, with an initial camera distance of 9. Perspective depth is concentrated near the camera, so a needlessly small near plane or distant far plane wastes precision. Orthographic depth is linear. The fixed inspection camera draws the 12 edges joining the main camera's eight world-space frustum corners.



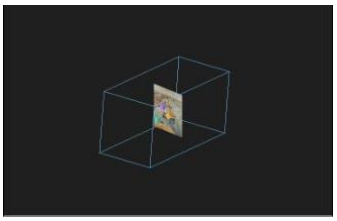
TRS / orthographic



SRT / orthographic



Perspective frustum



Orthographic frustum

The orthographic scene uses the same eye and target as the perspective views on page 1. Reference: WebGPU specification, Coordinate Systems (www.w3.org/TR/webgpu/#coordinate-systems).

Curves over the painting

Part D - Cubic Bezier and de Casteljau

I trace the lower edge of the boat with three cubic Bezier segments. The ten control points share the two joins. I evaluate the curve with these basis functions:

```
B(t) = b0(t)P0 + b1(t)P1 + b2(t)P2 + b3(t)P3
b0(t)=(1-t)^3       b1(t)=3(1-t)^2*t b2(t)=3(1-
t)*t^2    b3(t)=t^3
```

At t = 0 the weights are (1, 0, 0, 0), so B(0) = P0. At t = 1 they are (0, 0, 0, 1), so B(1) = P3. For de Casteljau I apply lerp(A, B, t) = (1-t)A + tB repeatedly: four points become three, then two, then one. The last point equals B(t). I can select the segment and change t in the panel.

C1 joins and tangent orientation

```
B'(t) = 3(1-t)^2(P1-P0) + 6(1-t)t(P2-P1) + 3t^2(P3-P2)
Q0 = P3
B'(1) = 3(P3-P2),  Q'(0) = 3(Q1-Q0)
C1 => Q1 = 2P3-P2
```

I reflect the opposite handle when a handle moves. Moving an anchor translates both adjacent handles. All segments have equal parameter intervals, so the global first derivatives also match. The initial join error is about 10^{-15}. The arrow uses atan2(tangent.y, tangent.x), with the tangent reversed on its return trip. Its speed is in curve-parameter units, not a constant world-space speed.

Catmull-Rom

```
C(t) = 0.5 * [2p1 + (-p0+p2)t
       + (2p0-5p1+4p2-p3)t^2
       + (-p0+3p1-3p2+p3)t^3]
```

This uniform segment runs from p1 to p2. At an interior join, both derivatives are (next-previous)/2. I duplicate the first and last points at the endpoints, giving three segments through four anchors. I keep this set of anchors separate from the Bezier handles.



Bezier trace with de Casteljau construction



Catmull-Rom through the four anchors

Shift + drag selects a control point; Edit points mode also allows dragging without Shift. I cast a mouse ray and intersect z = 0.04, parallel to the painting at z = 0, to avoid depth conflicts. Four clicks place new anchors. I export u = x/4.45 + 0.5 and v =
0.5-y/5.4, with u increasing rightward and v downward.

Integration and performance

Part E - One application

I connect all parts in one render loop. The Large view control swaps the main and inspection canvas sizes, while both cameras still render every frame. Reset scene restores node transforms, camera values, curve points, animation and controls. It also cancels an active drag. A master debug switch gates axes, control points, de Casteljau lines and the inspection frustum; the individual switches keep their settings.

Measurements

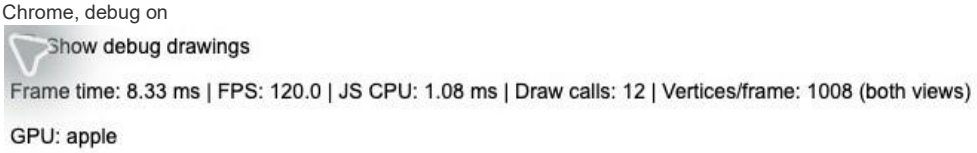
I measured the application in Google Chrome on macOS, using an Apple Silicon GPU and the local Vite server. Chrome version: 156.0.8078.80. Page zoom was 50%. The following are representative readings after warm-up, with the default perspective camera, boat curve and animations running; both views are included.

Configuration	Frame ms	FPS	JS CPU ms	Vertices	Draw calls
Debug on	8.26	121.1	0.99	1272	12
Debug off	8.33	120.0	1.08	1008	12

Frame time is the mean of the latest 120 requestAnimationFrame intervals; FPS is 1000 divided by that mean. JS CPU is a separate performance.now() measurement around the update, geometry generation, command submission and readouts. It does not wait for GPU completion. The display refreshes every 500 ms. These rolling readouts do not establish a speed improvement or an isolated GPU execution time.



Chrome, debug on



Chrome, debug off

I add the counts passed to actual draw() calls. Default: main = 6 painting + 108 cubes + 102 markers/arrow + 408 line vertices = 624; inspection adds 24 frustum vertices, giving 648. Total = 1272 vertices and 12 draw calls. With debug off: 2 * (6 + 108 + 6 + 384) = 1008. These are submitted vertices, including repeats.

Problems I checked

Mouse offset. Including the canvas border shifted the picked position. I subtract clientLeft/clientTop and use clientWidth/clientHeight for the content area. A test checks a known orthographic point.
Inspection clipping. The frustum was clipped when the inspection canvas became large in a narrow layout. I kept its eye and target fixed and expanded the orthographic volume from the transformed corner bounds.
Timing and reset. I use the raw frame interval for statistics and clamp only animation time. Returning from a hidden tab starts a fresh sample window; reset clears the scene and the measurement history.

Validation: 30 tests pass, including curve continuity, reset, picking and counts checked against issued draw commands. The Vite production build also completes. Source exports are reproduced on the next page.

Appendix - Normalized curve exports

I use u = 0 at the left and v = 0 at the top. The data below is taken from the three exported JSON files; only whitespace is shortened. Bezier joins share indices 3 and 6. Catmull-Rom duplicates endpoint neighbors when evaluating its first and last segments.
exports/boat-bezier.json


```
{
  "type": "bezier",
  "painting": "painting.jpg",
  "coordinates": {"u":"0 left, 1 right","v":"0 top, 1 bottom"},
  "controlPoints": [
    {"u":0.3,"v":0.59},
    {"u":0.34,"v":0.63},
    {"u":0.42,"v":0.65},
    {"u":0.48,"v":0.65},
    {"u":0.54,"v":0.65},
    {"u":0.64,"v":0.64},
    {"u":0.7,"v":0.62},
    {"u":0.76,"v":0.6},
```

```
    {"u":0.85,"v":0.57},
    {"u":0.91,"v":0.54}
  ],
  "segments": [[0,1,2,3],[3,4,5,6],[6,7,8,9]],
  "continuity": "C1",
  "endpointRule": "shared anchors and reflected handles",
  "parameterization": "equal segment intervals"
}
```


exports/boat-catmull-rom.json


```
{
  "type": "catmull-rom",
  "painting": "painting.jpg",
  "coordinates": {"u":"0 left, 1 right","v":"0 top, 1 bottom"},
  "controlPoints": [
    {"u":0.3,"v":0.59},
    {"u":0.48,"v":0.65},
    {"u":0.7,"v":0.62},
    {"u":0.91,"v":0.54}
  ],
  "segments": [[0,1],[1,2],[2,3]],
  "continuity": "C1",
  "endpointRule": "duplicate first and last points",
  "parameterization": "equal segment intervals"
}
```


exports/placed-bezier.json


```
{
  "type": "bezier",
  "painting": "painting.jpg",
  "coordinates": {"u":"0 left, 1 right","v":"0 top, 1 bottom"},
  "controlPoints": [
    {"u":0.3019043299517381,"v":0.5449801064110258},
    {"u":0.3316946936680212,"v":0.5547772454713527},
    {"u":0.4144323160944818,"v":0.5988672964661984},
    {"u":0.4806465122494364,"v":0.6037629407729868},
    {"u":0.5468607084043909,"v":0.6086585850797751},
    {"u":0.6278824357927185,"v":0.5924269182018483},
    {"u":0.6991895068814656,"v":0.574353972251756},
    {"u":0.7704965779702128,"v":0.5562810263016637},
    {"u":0.8736057001318434,"v":0.5084967162689865},
    {"u":0.9084889387819189,"v":0.49532526507243263}
  ],
  "segments": [[0,1,2,3],[3,4,5,6],[6,7,8,9]],
  "continuity": "C1",
  "endpointRule": "shared anchors and reflected handles",
  "parameterization": "equal segment intervals"
}
```


No generative AI tools were used for generating the core matrix, camera, or curve code. AI assistants were used strictly for debugging WebGPU validation errors, understanding syntax requirements, and formatting performance readouts.